

NAME: B. TARUNREDDY

REG NO : 192424213

COURSE CODE: DSA0515

**COURSE CODE : QUERY PROCESSING FOR DATA SCIENCE IN
SQL ENGINES**

LAB OUTPUTS : 21-40

DATE: 16-06-2026

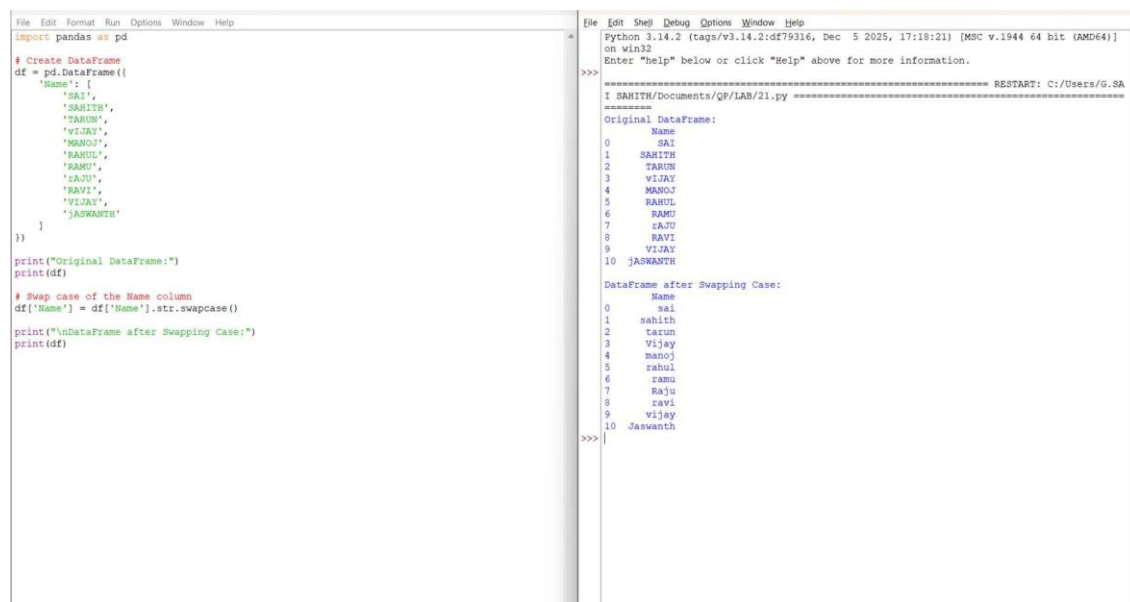
EXPERIMRNT-21:

Write a Pandas program to swap the cases of a specified character column in each Data Frame.

AIM: To write a Pandas program to swap the case of characters in a specified column of a DataFrame using the str.swapcase() function.

ALGORITHM:

1. Import the Pandas library.
2. Create a DataFrame containing a column named **Name** with the given names.
3. Display the original DataFrame.
4. Apply the str.swapcase() method to the **Name** column.
5. Store the modified values back into the **Name** column.
6. Display the DataFrame after swapping the case of the characters.
7. End the program.



```
File Edit Format Run Options Window Help
import pandas as pd

# Create DataFrame
df = pd.DataFrame({
    'Name': [
        'Sai',
        'SAHITH',
        'TARUN',
        'VIJAY',
        'MANOJ',
        'RAHUL',
        'RAMU',
        'RAJU',
        'RAVI',
        'VIJAY',
        'JASWANTH'
    ]
})

print("Original DataFrame:")
print(df)

# Swap case of the Name column
df['Name'] = df['Name'].str.swapcase()

print("\nDataFrame after Swapping Case:")
print(df)
```

Python 3.14.2 (tags/v3.14.2:df79316, Dec 5 2025, 17:18:21) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

===== RESTART: C:/Users/G.SA
1 SAHITH/Documents/QP/LAB/21.py =====

Original DataFrame:

	Name
0	Sai
1	SAHITH
2	TARUN
3	VIJAY
4	MANOJ
5	RAHUL
6	RAMU
7	RAJU
8	RAVI
9	VIJAY
10	JASWANTH

DataFrame after Swapping Case:

	Name
0	sai
1	sahith
2	tarun
3	viJay
4	manoj
5	rahul
6	ramu
7	raju
8	ravi
9	viJay
10	Jaswanth

Result :

The cases of all characters in the **Name** column are successfully swapped using the Pandas str.swapcase() method.

DATE: 16-06-2026

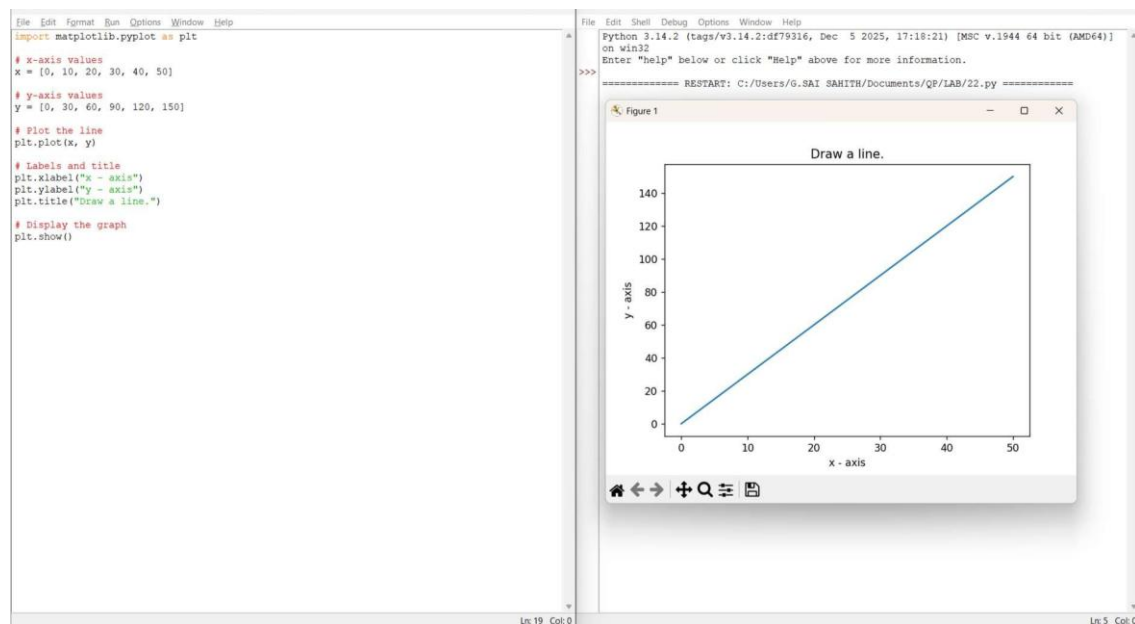
EXPERIMRNT-22:

Write a Python program to draw a line with suitable label in the x axis, y axis and a title.

AIM: To write a Python program using Matplotlib to draw a line graph with labels for the x-axis, y-axis, and a title.

ALGORITHM:

1. Import the matplotlib.pyplot module.
2. Create two lists containing x-axis and y-axis values.
3. Use the plot() function to draw the line.
4. Add a label to the x-axis using xlabel().
5. Add a label to the y-axis using ylabel().
6. Add a title using title().
7. Display the graph using show().



Result :

Thus, a line graph with suitable labels for the x-axis, y-axis, and a title was successfully drawn using Matplotlib.

DATE: 16-06-2026

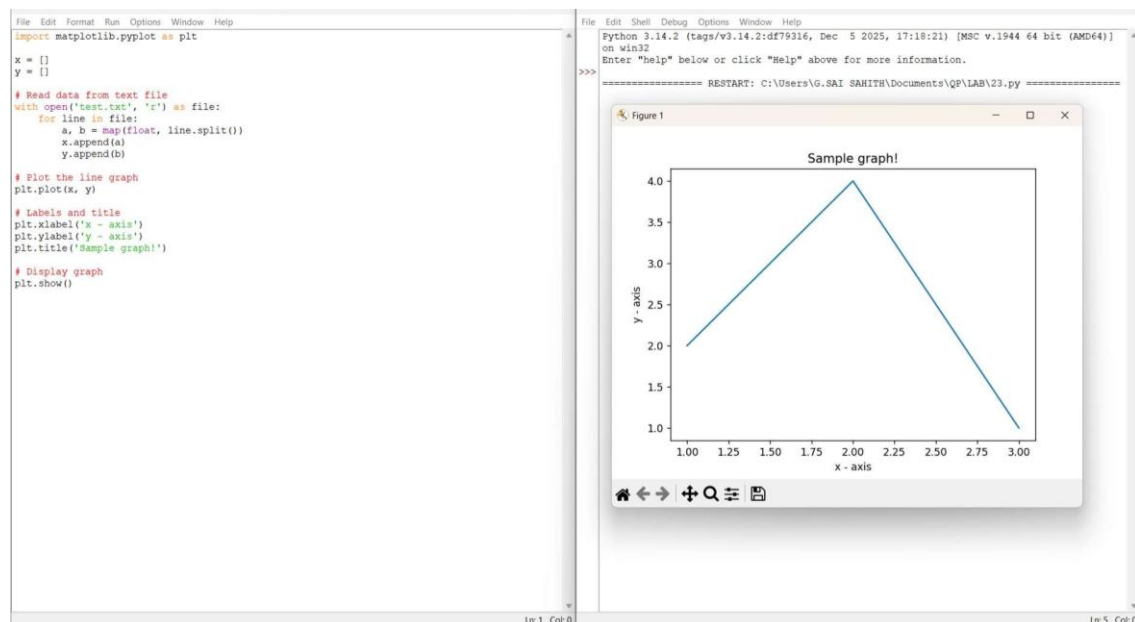
EXPERIMRNT-23:

Write a Python program to draw a line using given axis values taken from a text file, with suitable label in the x-axis, y-axis and a title.

AIM: To write a Python program that reads x-axis and y-axis values from a text file and draws a line graph with suitable labels and a title using Matplotlib.

ALGORITHM :

1. Create a text file named test.txt containing x and y values.
2. Import the matplotlib.pyplot module.
3. Open the text file and read the data.
4. Store the first column values in the x-axis list and the second column values in the y-axis list.
5. Use the plot() function to draw the line graph.
6. Add labels to the x-axis and y-axis.
7. Add a title to the graph.
8. Display the graph using show().
9. End the program.



Result :

Thus, the line graph was successfully plotted using the values (1,2), (2,4), and (3,1) with suitable labels and title.

DATE: 17-06-2026

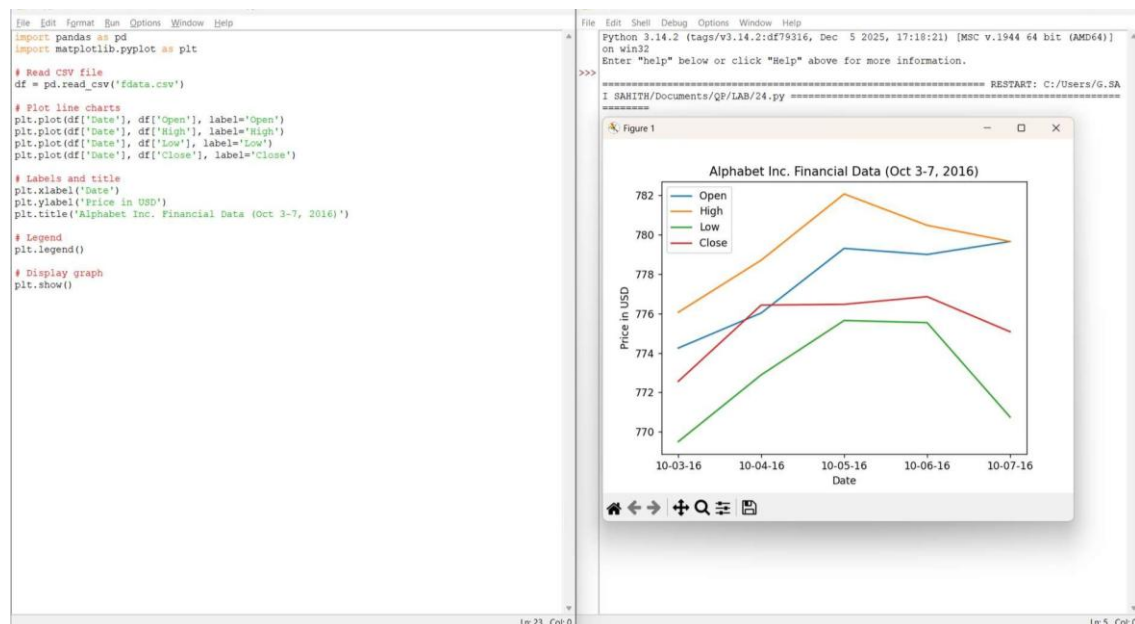
EXPERIMRNT-24:

24. Write a Python program to draw line charts of the financial data of Alphabetic. between October 3, 2016, 2016, ctober 7, 2016.

AIM: To write a Python program to plot the financial data of Alphabet Inc. from a CSV file and draw line charts for Open, High, Low, and Close prices using Matplotlib.

ALGORITHM:

1. Import the required libraries (pandas and matplotlib.pyplot).
2. Read the financial data from the CSV file (fdata.csv).
3. Extract the Date, Open, High, Low, and Close columns.
4. Plot line charts for Open, High, Low, and Close prices against Date.
5. Add labels to the x-axis and y-axis.
6. Add a suitable title.
7. Display the legend.
8. Show the graph.
9. End the program.



Result :

Thus, the line charts representing the Open, High, Low, and Close stock prices of Alphabet Inc. from October 3, 2016 to October 7, 2016, were successfully plotted .

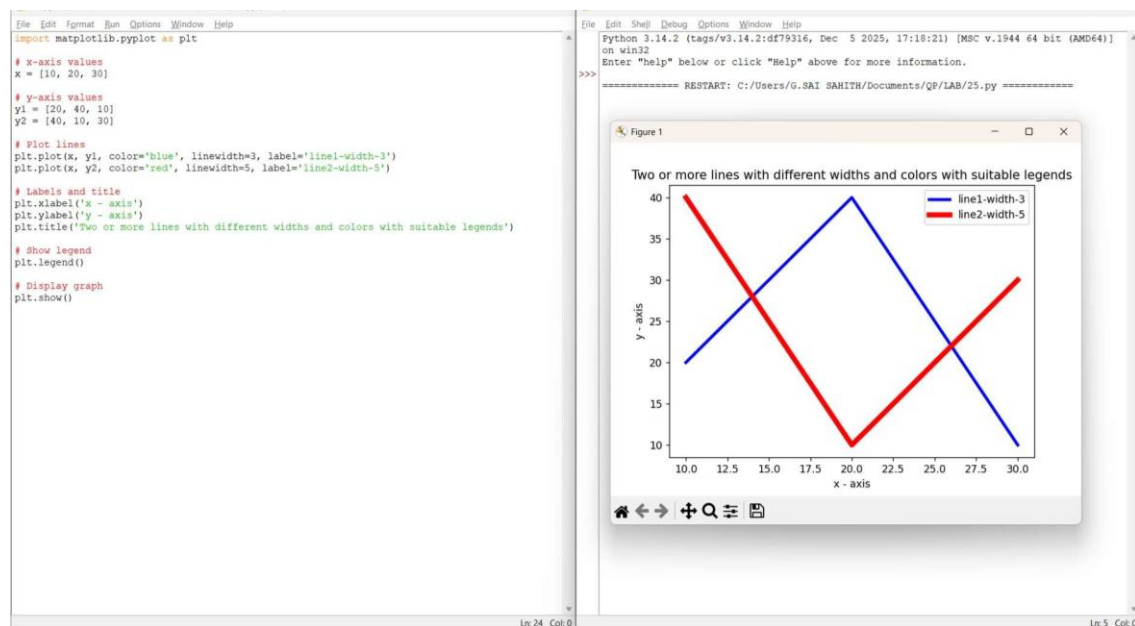
DATE: 17-06-2026

EXPERIMENT-25:

AIM: To write a Python program to plot two or more lines with different colours, line widths, and legends using Matplotlib.

ALGORITHM:

1. Import the matplotlib.pyplot module.
2. Create lists for x-axis and y-axis values.
3. Plot the first line using a specific colour and line width.
4. Plot the second line using a different colour and line width.
5. Add labels for the x-axis and y-axis.
6. Add a suitable title to the graph.
7. Display the legend.
8. Show the graph using show().
9. End the program.



Result :

Thus, the Python program to plot two or more lines with different colours, widths, and legends was executed successfully using Matplotlib.

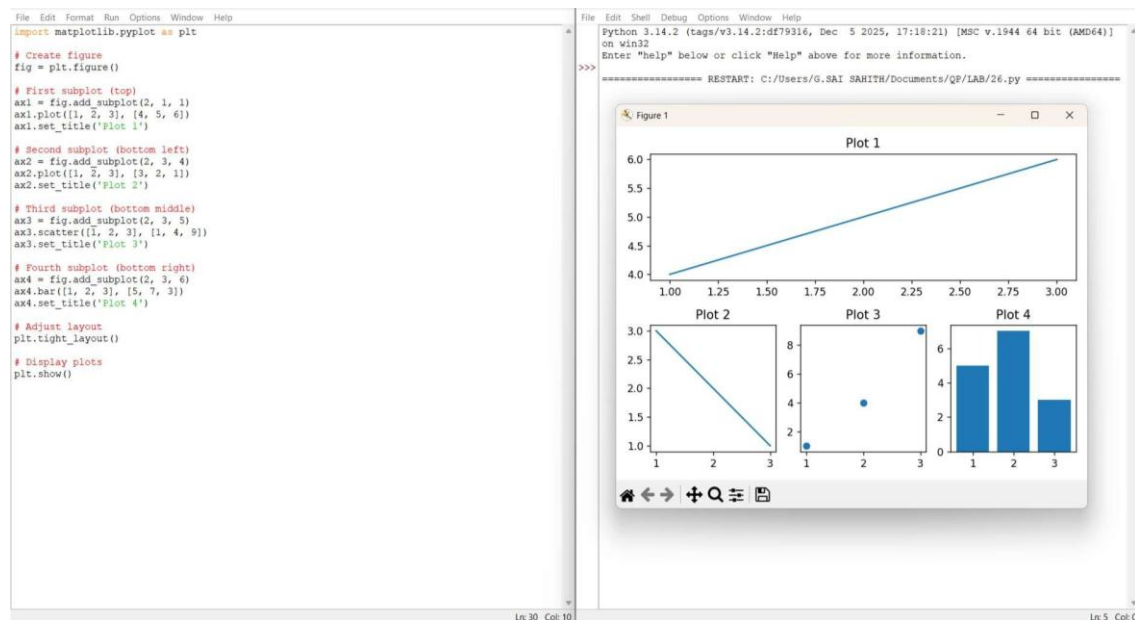
DATE: 17-06-2026

EXPERIMRNT-26:

AIM: To write a Python program to create multiple plots using Matplotlib.

ALGORITHM:

1. Import the matplotlib.pyplot module.
2. Create a figure using figure().
3. Add multiple subplots using add_subplot().
4. Plot different graphs in each subplot.
5. Add titles to the subplots.
6. Adjust the layout using tight_layout().
7. Display all plots using show().
8. End the program.



Result :

Thus, the Python program to create multiple plots using Matplotlib was executed successfully.

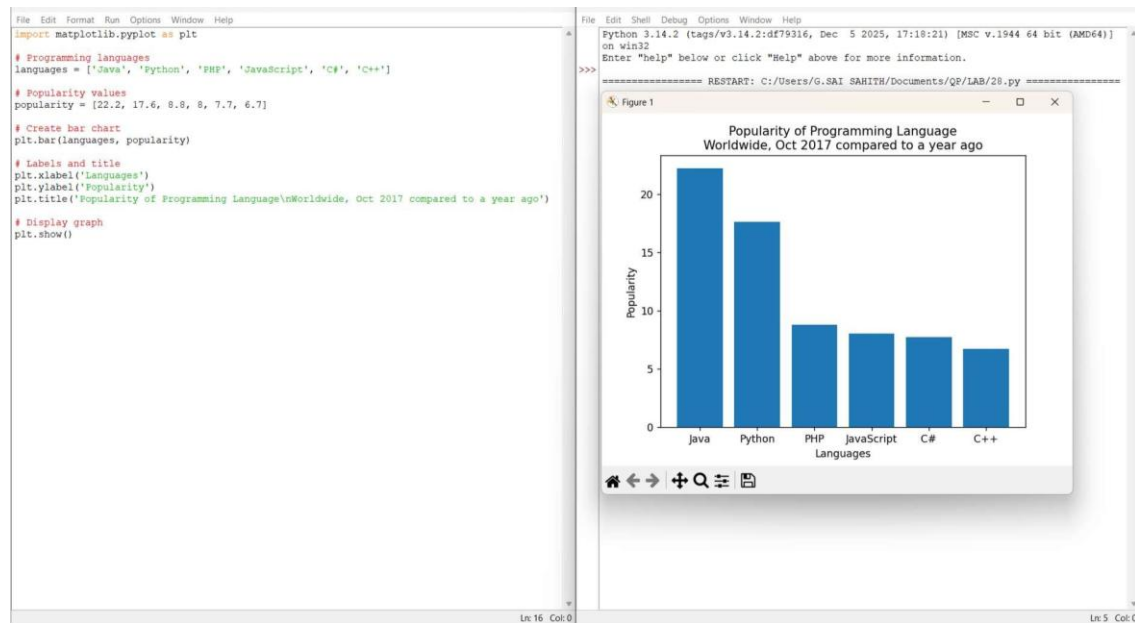
DATE: 18-06-2026

EXPERIMENT-27:

AIM: To write a Python program to display a bar chart representing the popularity of various programming languages using Matplotlib.

ALGORITHM:

1. Import the matplotlib.pyplot module.
2. Create a list of programming languages.
3. Create a list of popularity values corresponding to each language.
4. Use the bar() function to create the bar chart.
5. Add labels to the x-axis and y-axis.
6. Add a suitable title to the chart.
7. Display the chart using show().
8. End the program.



Result:

Thus, the bar chart representing the popularity of different programming languages was successfully displayed using Matplotlib.

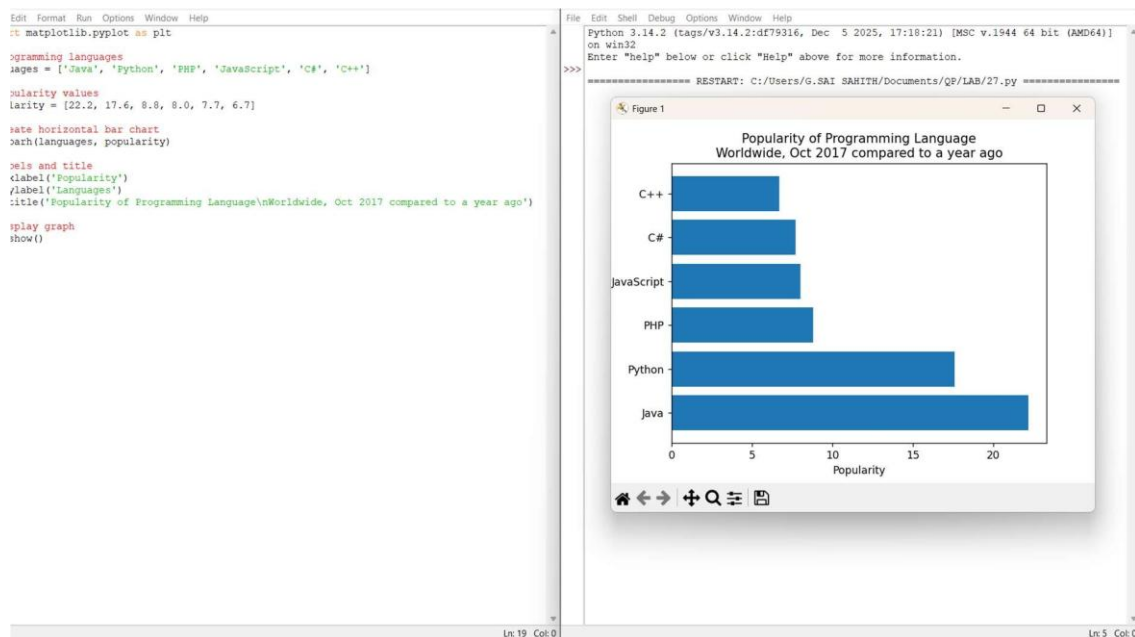
DATE: 18-06-2026

EXPERIMENT-28:

AIM: To write a Python program to display a horizontal bar chart representing the popularity of programming languages using Matplotlib.

ALGORITHM:

1. Import the matplotlib.pyplot module.
2. Create a list of programming languages.
3. Create a list of popularity values.
4. Use the barh() function to create a horizontal bar chart.
5. Add labels to the x-axis and y-axis.
6. Add a suitable title to the chart.
7. Display the chart using show().
8. End the program.



Result:

Thus, the horizontal bar chart representing the popularity of programming languages was successfully displayed using Matplotlib.

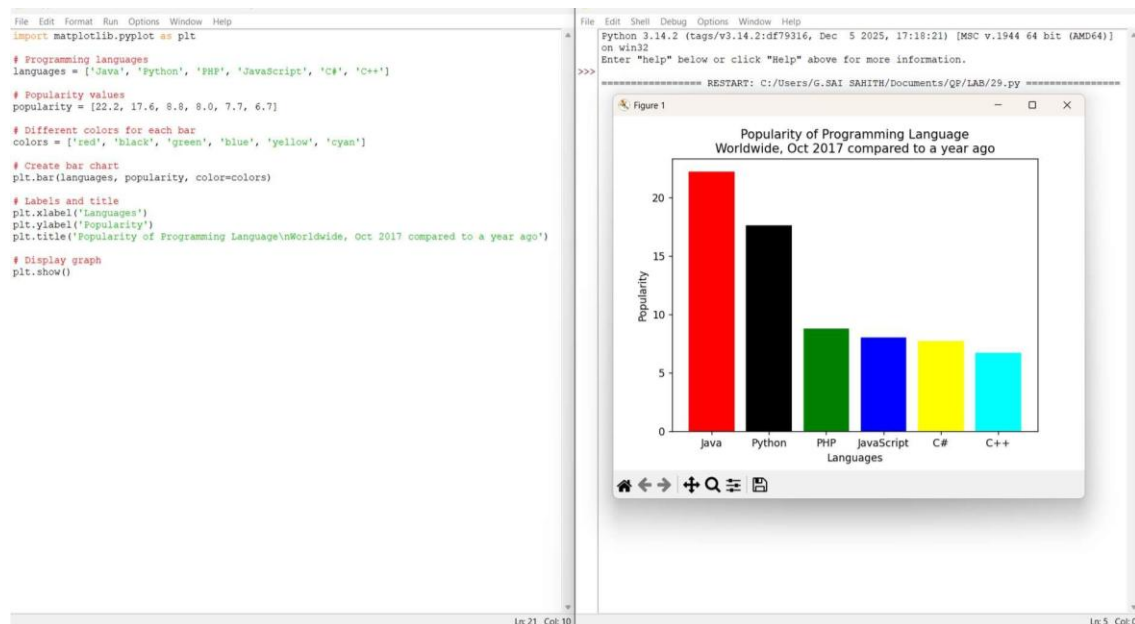
DATE: 18-06-2026

EXPERIMRNT-29:

AIM: To write a Python program to display a bar chart of the popularity of programming languages using different colours for each bar.

ALGORITHM:

1. Import the matplotlib.pyplot module.
2. Create a list of programming languages.
3. Create a list of popularity values.
4. Create a list of different colours for each bar.
5. Use the bar() function to draw the bar chart with collars.
6. Add labels to the x-axis and y-axis.
7. Add a suitable title.
8. Display the chart using show().
9. End the program.



Result:

Thus, the bar chart displaying the popularity of programming languages with different colours for each bar was successfully created using Matplotlib.

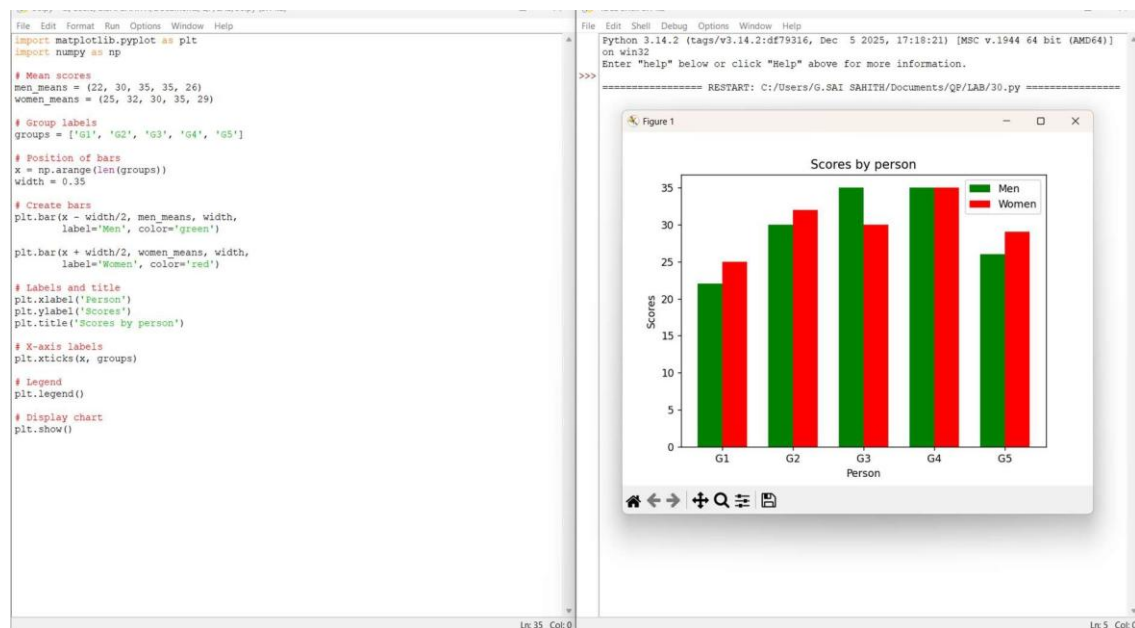
DATE: 18-06-2026

EXPERIMRNT-30:

AIM: To write a Python program to create a grouped bar chart showing scores of men and women for different groups using Matplotlib.

ALGORITHM:

1. Import the required libraries (matplotlib.pyplot and NumPy).
2. Create lists containing the scores of men and women.
3. Create group labels (G1, G2, G3, G4, G5).
4. Set the positions of bars using NumPy.
5. Plot bars for men and women with different colours.
6. Add labels for the x-axis and y-axis.
7. Add a title and legend.
8. Display the chart using show().
9. End the program.



Result:

Thus, the grouped bar chart showing scores by group and gender was successfully created using Matplotlib.

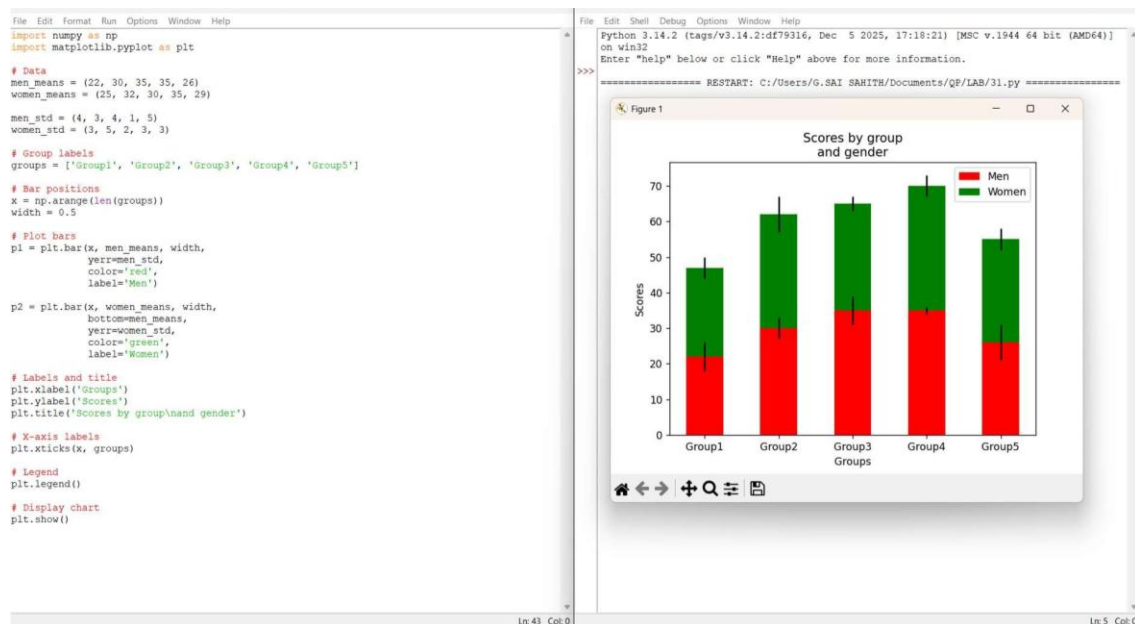
DATE :19-06-2026

EXPERIMENT-31

AIM: To write a Python program to create a stacked bar chart with error bars for men and women scores using Matplotlib.

ALGORITHM:

1. Import the required libraries (matplotlib.pyplot and numpy).
2. Define the mean scores of men and women.
3. Define the standard deviations for men and women.
4. Create group labels and bar positions.
5. Plot the men's bars with error bars.
6. Plot the women's bars on top of the men's bars using the bottom parameter.
7. Add labels, title, and legend.
8. Display the chart using show().
9. End the program.



Result:

Thus, the stacked bar chart with error bars for men and women scores was successfully created using Matplotlib.

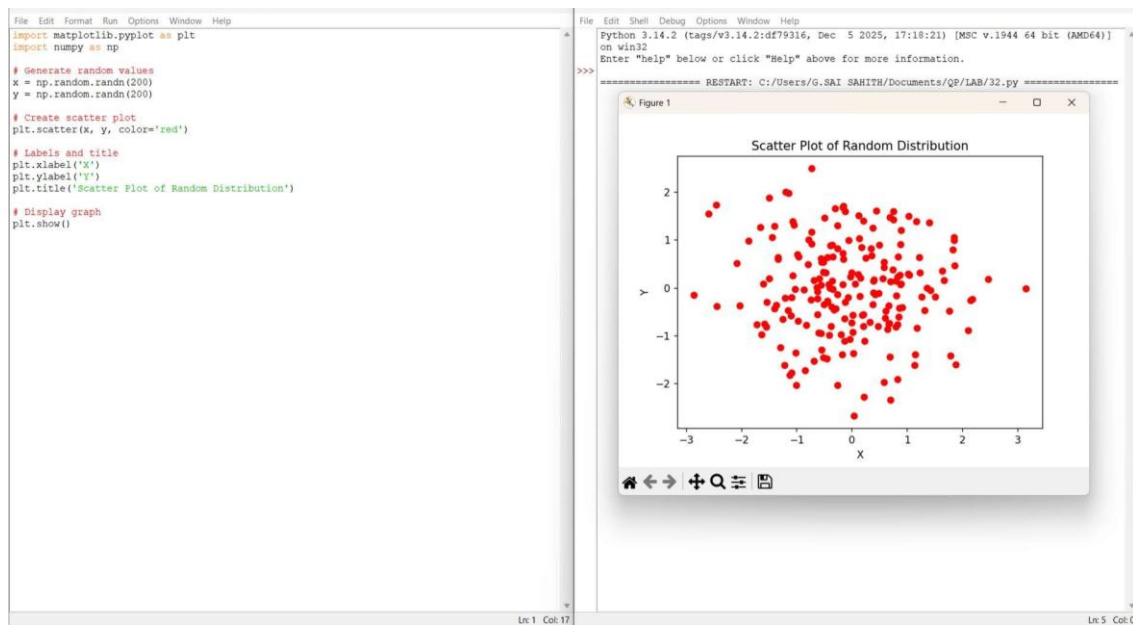
DATE :19-06-2026

EXPERIMENT-32:

AIM: To write a Python program to create a scatter plot using randomly generated values for X and Y using Matplotlib.

ALGORITHM:

1. Import the required libraries (matplotlib.pyplot and NumPy).
2. Generate random values for X and Y coordinates.
3. Use the scatter() function to plot the points.
4. Label the x-axis and y-axis.
5. Add a title to the graph.
6. Display the scatter plot using show().
7. End the program.



Result:

Thus, the scatter graph of randomly generated X and Y distributions was successfully plotted using Matplotlib.

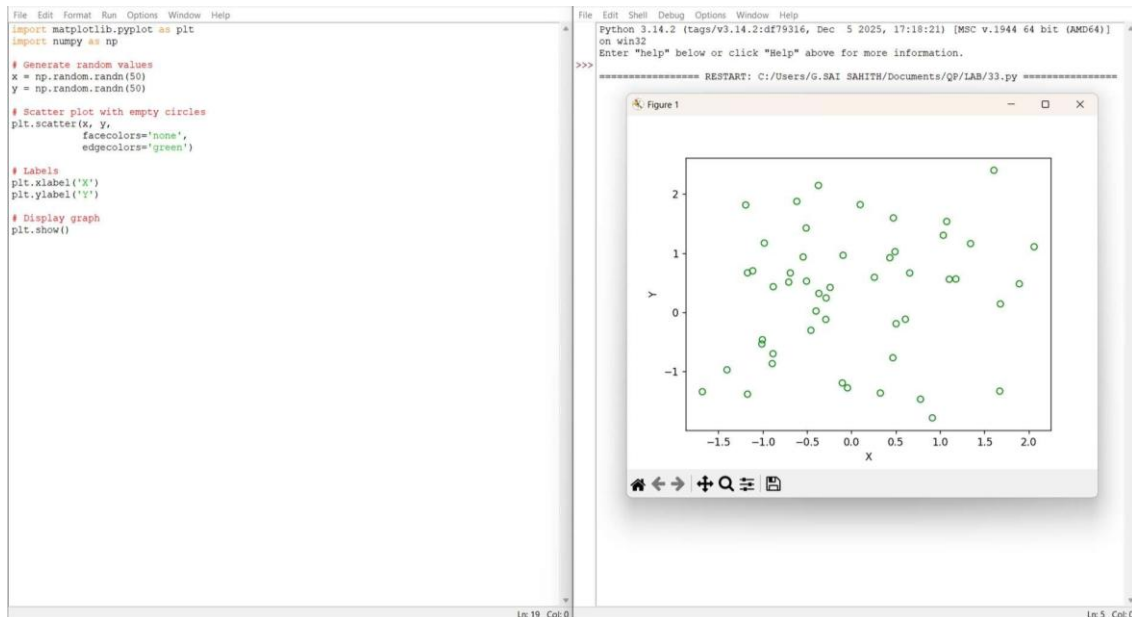
DATE :19-06-2026

EXPERIMENT-33:

AIM: To write a Python program to create a scatter plot with empty circles using random distributions in X and Y.

ALGORITHM:

1. Import the required libraries (matplotlib.pyplot and NumPy).
2. Generate random values for X and Y coordinates.
3. Use the scatter() function to plot the points.
4. Set forecolours='none' to create empty circles.
5. Set decolours='green' to display circle borders.
6. Label the x-axis and y-axis.
7. Display the scatter plot using show().
8. End the program.



Result:

Thus, the scatter plot with empty circles using random distributions in X and Y was successfully created using Matplotlib.

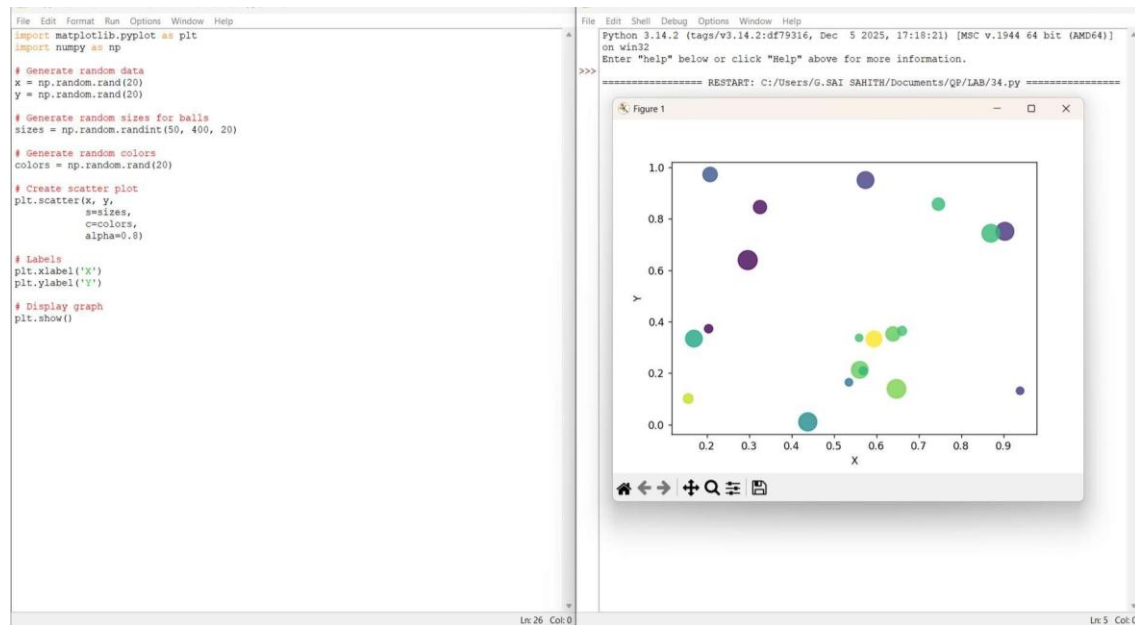
DATE :22-06-2026

EXPERIMENT-34:

AIM: To write a Python program to create a scatter plot with randomly generated points and balls of different sizes using Matplotlib.

ALGORITTHAM:

1. Import the required libraries (matplotlib.pyplot and NumPy).
2. Generate random values for X and Y coordinates.
3. Generate random values for marker sizes.
4. Generate random colours for the markers.
5. Use the scatter() function to create the scatter plot.
6. Label the x-axis and y-axis.
7. Display the graph using show().
8. End the program.



Result:

Thus, the scatter plot with randomly generated balls of different sizes was successfully created using Matplotlib.

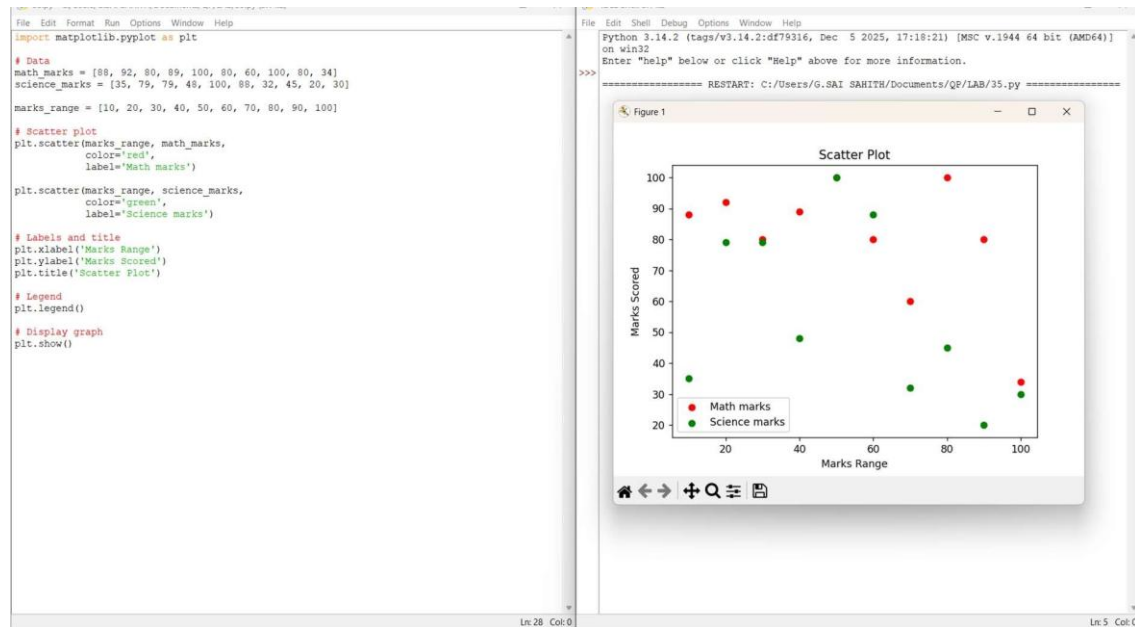
DATE :22-06-2026

EXPERIMENT-35:

AIM: To write a Python program to draw a scatter plot comparing Mathematics and Science marks of 10 students using Matplotlib.

ALGORITHM:

1. Import the matplotlib.pyplot module.
2. Create lists for Mathematics marks, Science marks, and marks range.
3. Plot Mathematics marks using the scatter() function.
4. Plot Science marks using another scatter() function.
5. Add labels for x-axis and y-axis.
6. Add a title and legend.
7. Display the scatter plot using show().
8. End the program.



Result:

Thus, the scatter plot comparing Mathematics and Science marks of 10 students was successfully created using Matplotlib.

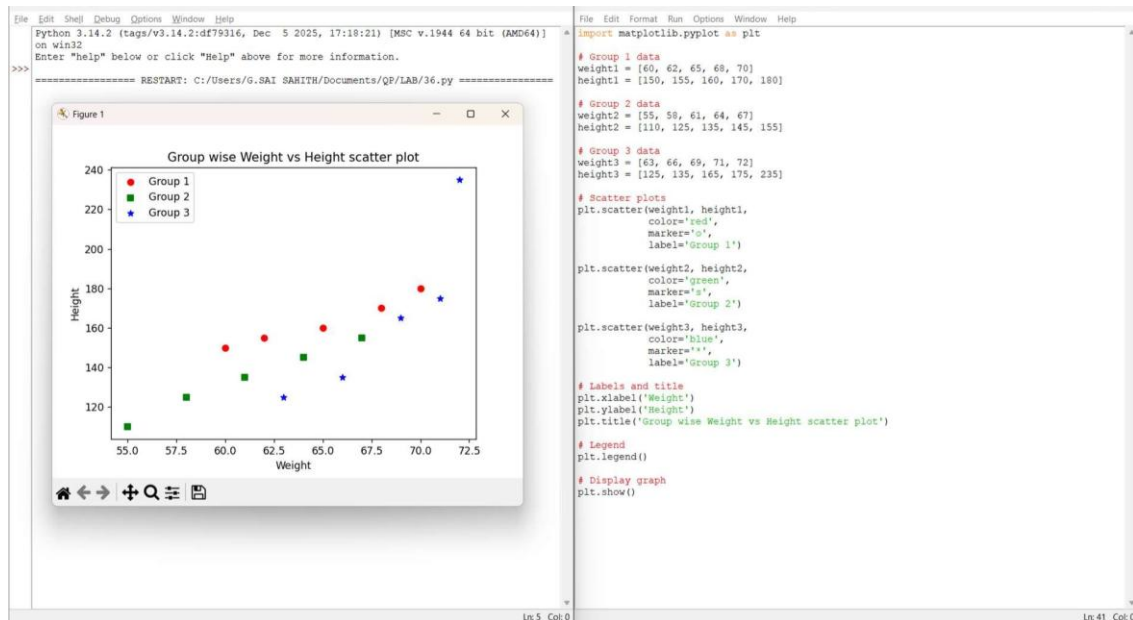
DATE :22-06-2026

EXPERIMENT-36:

AIM: To write a Python program to draw a scatter plot for three different groups comparing weights and heights using Matplotlib.

ALGORITHM:

1. Import the matplotlib.pyplot module.
2. Create separate weight and height data for three groups.
3. Plot each group using the scatter() function with different colours and markers.
4. Add labels for the x-axis and y-axis.
5. Add a title to the graph.
6. Add a legend to identify the groups.
7. Display the scatter plot using show().
8. End the program.



Result:

Thus, the scatter plot for three different groups comparing weights and heights was successfully created using Matplotlib.

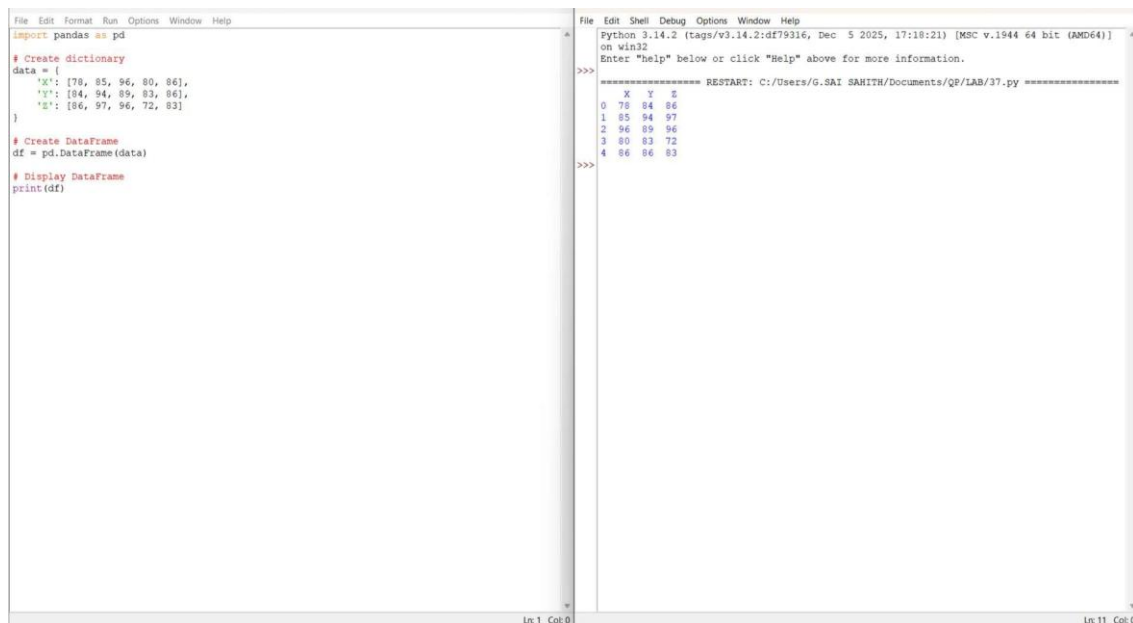
DATE :23-06-2026

EXPERIMENT-37:

AIM: To write a Pandas program to create a Data Frame from a dictionary and display the Data Frame.

ALGORITHM:

1. Import the Pandas library.
2. Create a dictionary containing the data.
3. Convert the dictionary into a Data Frame using `pd.DataFrame()`.
4. Display the Data Frame using the `print()` function.
5. End the program.



The screenshot displays a Python IDE with two windows. The left window shows the source code for creating a DataFrame from a dictionary. The right window shows the execution output, which includes the restart path and the resulting DataFrame table.

```
File Edit Format Run Options Window Help
import pandas as pd

# Create dictionary
data = {
    'X': [78, 85, 96, 80, 86],
    'Y': [84, 94, 89, 83, 86],
    'Z': [86, 97, 96, 72, 83]
}

# Create DataFrame
df = pd.DataFrame(data)

# Display DataFrame
print(df)
```

Python 3.14.2 (tags/v3.14.2:df79316, Dec 5 2025, 17:18:21) [MSC v.1944 64 bit (AMD64)]
on win32
Enter "help" below or click "Help" above for more information.
===== RESTART: C:/Users/G.SAI SAHITH/Documents/Q&LAB/37.py =====

	X	Y	Z
0	78	84	86
1	85	94	97
2	96	89	96
3	80	83	72
4	86	86	83

Result:

Thus, the Data Frame was successfully created from the given dictionary and displayed using Pandas.

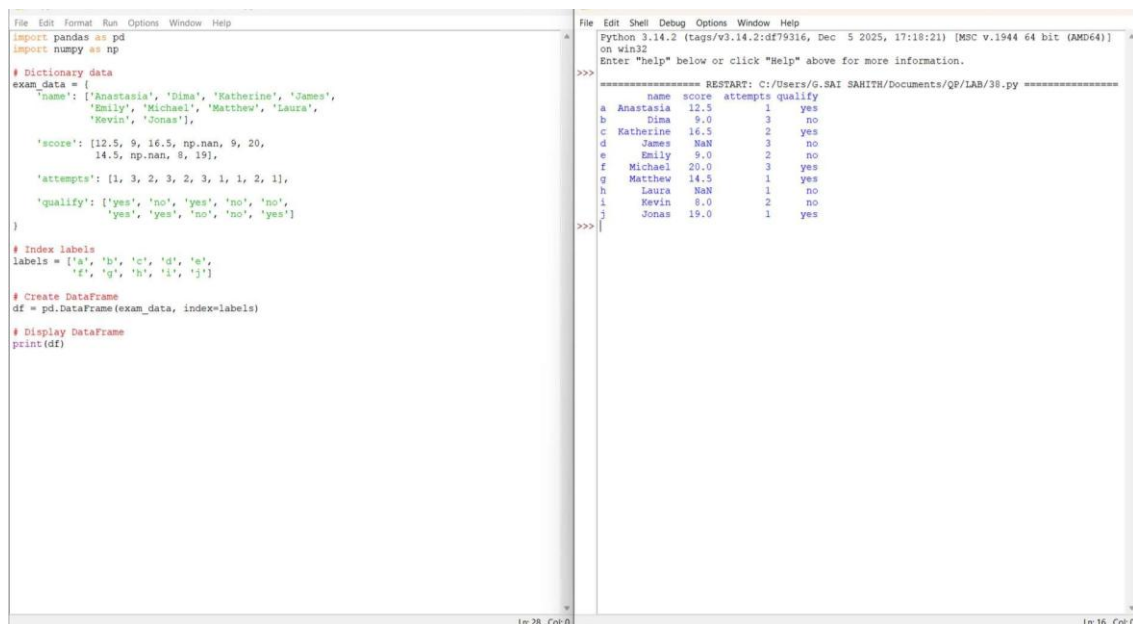
DATE :23-06-2026

EXPERIMENT-38:

AIM: To write a Pandas program to create and display a Data Frame from a specified dictionary using custom index labels.

ALGORITHM:

1. Import the Pandas and NumPy libraries.
2. Create a dictionary containing student information.
3. Create a list of index labels.
4. Use `pd.DataFrame()` to create a Data Frame with the specified index labels.
5. Display the Data Frame using the `print()` function.
6. End the program.



```
File Edit Format Run Options Window Help
import pandas as pd
import numpy as np

# Dictionary data
exam_data = {
    'name': ['Anastasia', 'Dima', 'Katherine', 'James',
            'Emily', 'Michael', 'Matthew', 'Laura',
            'Kevin', 'Jonas'],
    'score': [12.5, 9, 16.5, np.nan, 9, 20,
            14.5, np.nan, 8, 19],
    'attempts': [1, 3, 2, 3, 2, 3, 1, 1, 2, 1],
    'qualify': ['yes', 'no', 'yes', 'no', 'no',
            'yes', 'yes', 'no', 'no', 'yes']
}

# Index labels
labels = ['a', 'b', 'c', 'd', 'e',
        'f', 'g', 'h', 'i', 'j']

# Create DataFrame
df = pd.DataFrame(exam_data, index=labels)

# Display DataFrame
print(df)
```

Python 3.14.2 (tags/v3.14.2:df79316, Dec 5 2025, 17:18:21) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

```
>>>
===== RESTART: C:/Users/G.SAI SAKITH/Documents/QB/LAB/38.py =====
a  name  score  attempts  qualify
b  Dima   9.0        3      no
c  Katherine 16.5        2      yes
d  James  NaN        3      no
e  Emily   9.0        2      no
f  Michael 20.0        3      yes
g  Matthew 14.5        1      yes
h  Laura  NaN        1      no
i  Kevin   8.0        2      no
j  Jonas  19.0        1      yes
>>>
```

Ln: 28 Col: 0

Ln: 16 Col: 0

Result:

Thus, the Data Frame was successfully created from the specified dictionary data using custom index labels and displayed using Pandas.

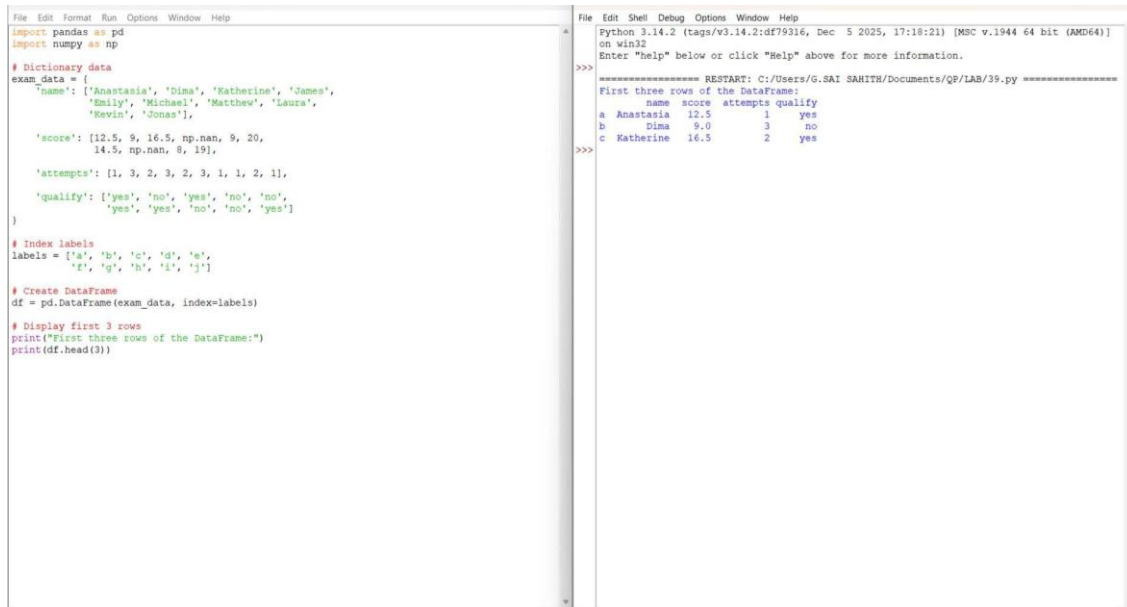
DATE :23-06-2026

EXPERIMENT-39:

AIM: To write a Pandas program to display the first three rows of a Data Frame using the head() function.

ALGORITHM:

1. Import the Pandas and NumPy libraries.
2. Create a dictionary containing student data.
3. Create index labels.
4. Convert the dictionary into a Data Frame.
5. Use the head(3) function to retrieve the first three rows.
6. Display the result.
7. End the program.



```
File Edit Format Run Options Window Help
import pandas as pd
import numpy as np

# Dictionary data
exam_data = {
    'name': ['Anastasia', 'Dima', 'Katherine', 'James',
            'Emily', 'Michael', 'Matthew', 'Laura',
            'Sevin', 'Jonas'],
    'score': [12.5, 9, 16.5, np.nan, 9, 20,
            14.5, np.nan, 8, 19],
    'attempts': [1, 3, 2, 3, 2, 3, 1, 1, 2, 1],
    'qualify': ['yes', 'no', 'yes', 'no', 'no',
            'yes', 'yes', 'no', 'no', 'yes']
}

# Index labels
labels = ['a', 'b', 'c', 'd', 'e',
        'f', 'g', 'h', 'i', 'j']

# Create DataFrame
df = pd.DataFrame(exam_data, index=labels)

# Display first 3 rows
print("First three rows of the DataFrame:")
print(df.head(3))
```

Python 3.14.2 (tags/v3.14.2:df79316, Dec 5 2025, 17:18:21) [MSC v.1944 64 bit (AMD64)]
on win32
Enter "help" below or click "Help" above for more information.

```
>>>
===== RESTART: C:/Users/G.SAI SAKITH/Documents/QP/LAB/39.py =====
First three rows of the DataFrame:
   name  score  attempts  qualify
a  Anastasia   12.5         1     yes
b      Dima     9.0         3     no
c  Katherine   16.5         2     yes
>>>
```

Result:

Thus, the first three rows of the given Data Frame were successfully displayed using the head(3) function in Pandas.

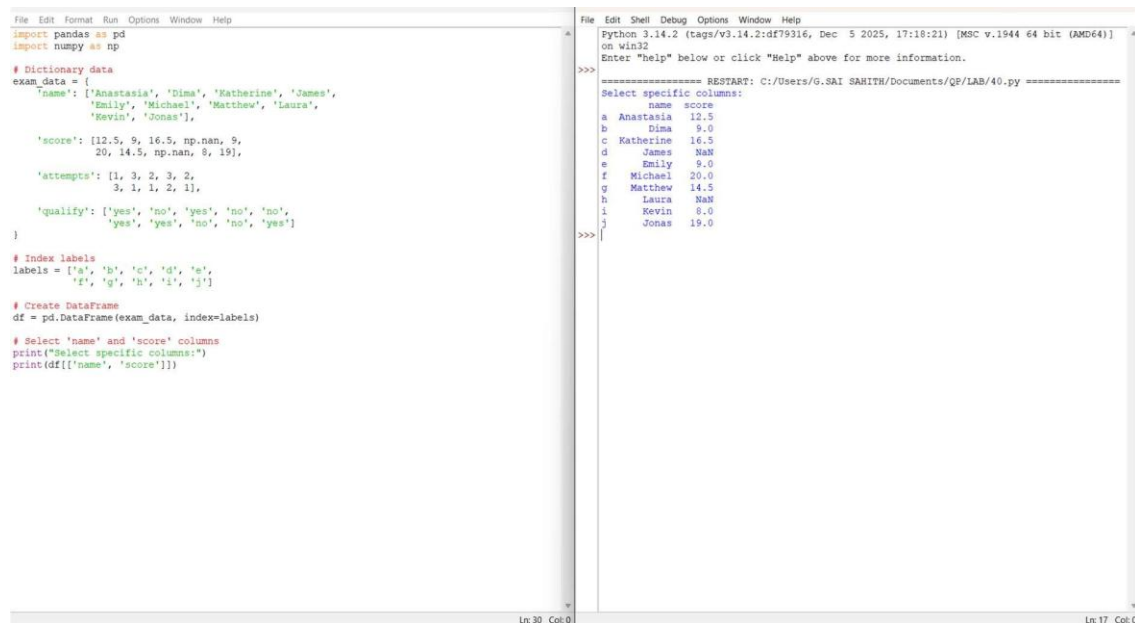
DATE :23-06-2026

EXPERIMENT-40:

AIM: To write a Pandas program to select and display only the 'name' and 'score' columns from a Data Frame.

ALGORITHM:

1. Import the Pandas and NumPy libraries.
2. Create a dictionary containing student data.
3. Create index labels.
4. Convert the dictionary into a Data Frame.
5. Select the 'name' and 'score' columns using column indexing.
6. Display the selected columns.
7. End the program.



The screenshot shows a Python IDE with two panes. The left pane contains the source code, and the right pane shows the output of the program.

```
File Edit Format Run Options Window Help
import pandas as pd
import numpy as np

# Dictionary data
exam_data = {
    'name': ['Anastasia', 'Dima', 'Katherine', 'James',
            'Emily', 'Michael', 'Matthew', 'Laura',
            'Kevin', 'Jonas'],
    'score': [12.5, 9, 16.5, np.nan, 9,
            20, 14.5, np.nan, 8, 19],
    'attempts': [1, 3, 2, 3, 2,
                3, 1, 1, 2, 1],
    'qualify': ['yes', 'no', 'yes', 'no', 'no',
               'yes', 'yes', 'no', 'no', 'yes']
}

# Index labels
labels = ['a', 'b', 'c', 'd', 'e',
          'f', 'g', 'h', 'i', 'j']

# Create DataFrame
df = pd.DataFrame(exam_data, index=labels)

# Select 'name' and 'score' columns
print("Select specific columns:")
print(df[['name', 'score']])
```

Python 3.14.2 (tags/v3.14.2:df79316, Dec 5 2025, 17:18:21) [MSC v.1944 64 bit (AMD64)]
on win32
Enter "help" below or click "Help" above for more information.

```
>>>
===== RESTART: C:/Users/G.SAI SAKITH/Documents/QP/LAB/40.py =====
Select specific columns:
   name  score
a  Anastasia  12.5
b      Dima    9.0
c  Katherine  16.5
d      James  NaN
e      Emily    9.0
f    Michael  20.0
g    Matthew  14.5
h      Laura  NaN
i      Kevin    8.0
j      Jonas  19.0
>>>
```

Ln: 30 Col: 0

Ln: 17 Col: 0

Result:

Thus, the 'name' and 'score' columns were successfully selected and displayed from the Data Frame using Pandas.